

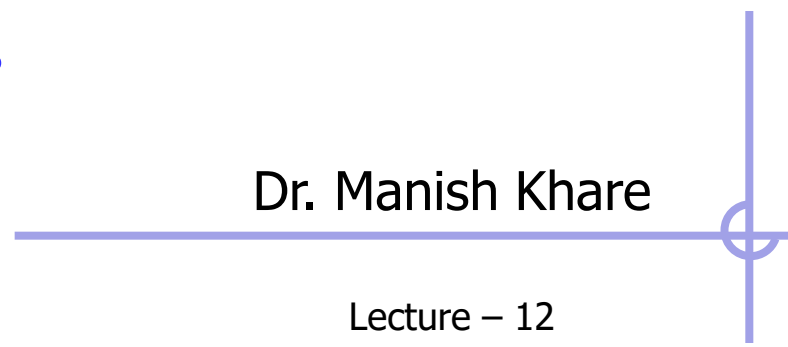
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture – 12



Type of Queue

➤ A queue data structure can be classified into the following types:

- 1. Circular Queue
- 2. Dequeue
- 3. Priority Queue

Problems in Queue

- In a normal Queue Data Structure, we can insert elements until queue becomes full. But once if queue becomes full, we can not insert the next element until all the elements are deleted from the queue.
- For example consider the queue below.
- After inserting all the elements into the queue.

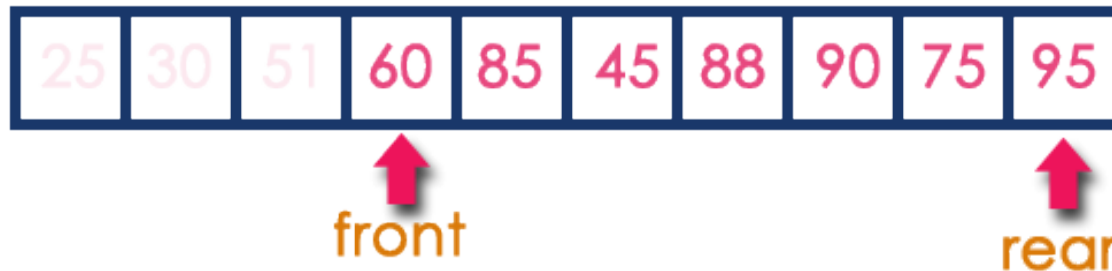
Queue is Full



➤ Now consider the following situation

- Now consider the following situation

Queue is Full (Even three elements are deleted)

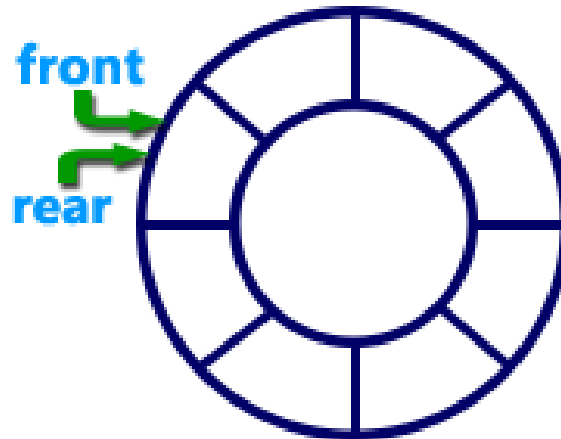


- This situation also says that Queue is Full and we can not insert the new element because, '**rear**' is still at last position. In above situation, even though we have empty positions in the queue we can not make use of them to insert new element. This is the major problem in normal queue data structure.

- 
- To resolve this problem, we have two solutions. First, shift the elements to the left so that the vacant space can be occupied and utilized efficiently. But this can be very time-consuming, especially when the queue is quite large.
 - The second option is to use a circular queue. In the circular queue, the first index comes right after the last index.

Circular Queue

- Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.



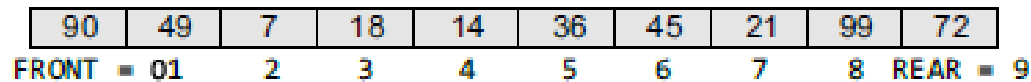
- 
- The circular queue will be full only when $\text{front} = 0$ and $\text{rear} = \text{Max} - 1$.
 - A circular queue is implemented in the same manner as a linear queue is implemented.
 - The only difference will be in the code that performs insertion and deletion operations.

Implementation of Circular Queue

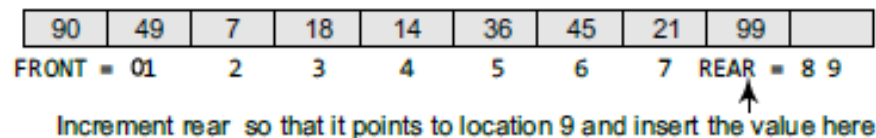
- To implement a circular queue data structure using array, we first perform the following steps before we implement actual operations.
 - **Step 1:** Include all the **header files** which are used in the program and define a constant '**SIZE**' with specific value.
 - **Step 2:** Declare all **user defined functions** used in circular queue implementation.
 - **Step 3:** Create a one dimensional array with above defined SIZE (**int cQueue[SIZE]**)
 - **Step 4:** Define two integer variables '**front**' and '**rear**' and initialize both with '**-1**'. (**int front = -1, rear = -1**)
 - **Step 5:** Implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on circular queue.

➤ For insertion, we now have to check for the following three conditions:

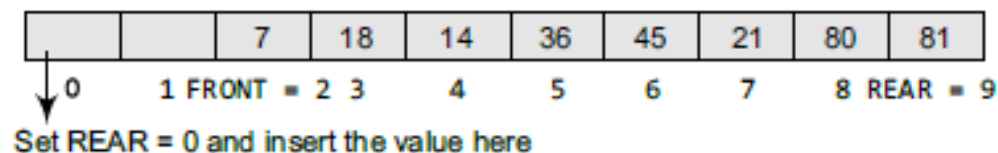
- If $\text{front} = 0$ and $\text{rear} = \text{MAX} - 1$, then the circular queue is full. Look at the queue given in below Fig. which illustrates this point.



- If $\text{rear} \neq \text{MAX} - 1$, then rear will be incremented and the value will be inserted as illustrated in below Fig.



- If $\text{front} \neq 0$ and $\text{rear} = \text{MAX} - 1$, then it means that the queue is not full. So, set $\text{rear} = 0$ and insert the new element there, as shown in following Fig.



enQueue(value) - Inserting value into the Circular Queue

Step 1: IF FRONT = 0 and Rear = MAX - 1
 Write "OVERFLOW"
 Goto step 4
 [End OF IF]

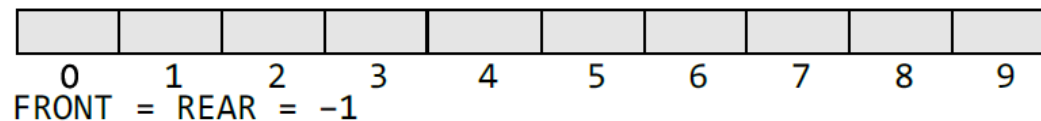
Step 2: IF FRONT = -1 and REAR = -1
 SET FRONT = REAR = 0
 ELSE IF REAR = MAX - 1 and FRONT != 0
 SET REAR = 0
 ELSE
 SET REAR = REAR + 1
 [END OF IF]

Step 3: SET QUEUE[REAR] = VAL

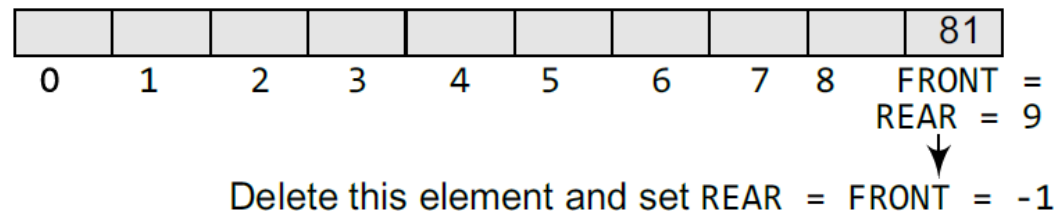
Step 4: EXIT

➤ To delete an element, again we check for three conditions.

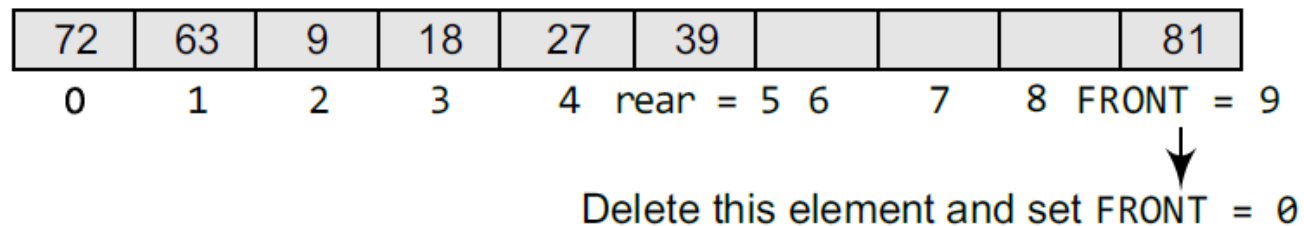
- Look at following Fig. If $\text{front} = -1$, then there are no elements in the queue. So, an underflow condition will be reported.



- If the queue is not empty and $\text{front} = \text{rear}$, then after deleting the element at the front the queue becomes empty and so front and rear are set to -1 . This is illustrated in following fig.



- If the queue is not empty and $\text{front} = \text{MAX}-1$, then after deleting the element at the front, front is set to 0. This is shown in Fig.



deQueue() - Deleting a value from the Circular Queue

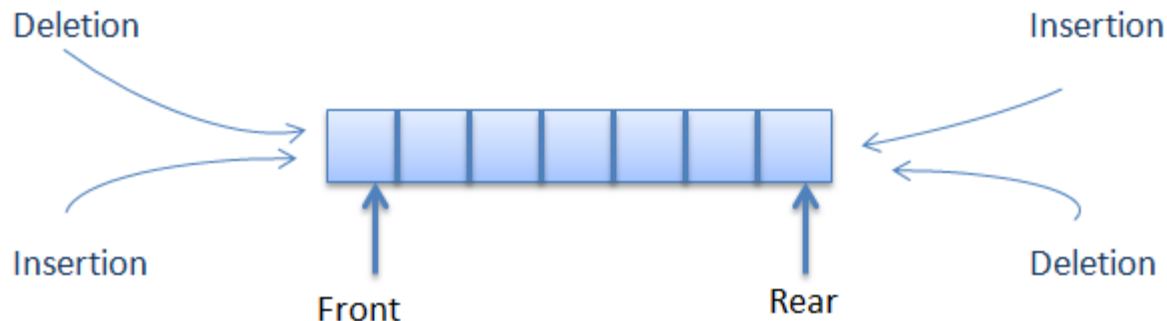
```
Step 1: IF FRONT = -1
        Write "UNDERFLOW"
        Goto Step 4
    [END of IF]
Step 2: SET VAL = QUEUE[FRONT]
Step 3: IF FRONT = REAR
        SET FRONT = REAR = -1
    ELSE
        IF FRONT = MAX - 1
            SET FRONT = 0
        ELSE
            SET FRONT = FRONT + 1
        [END of IF]
    [END OF IF]
Step 4: EXIT
```

display() - Displays the elements of a Circular Queue

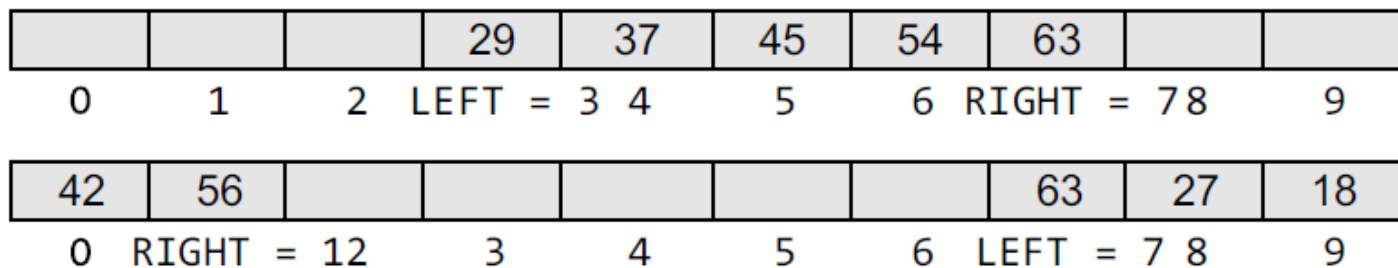
- We can use the following steps to display the elements of a circular queue...
- **Step 1:** Check whether **queue** is **EMPTY**. (**front == -1**)
- **Step 2:** If it is **EMPTY**, then display "**Queue is EMPTY!!!**" and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then define an integer variable '**i**' and set '**i = front**'.
- **Step 4:** Check whether '**front <= rear**', if it is **TRUE**, then display '**queue[i]**' value and increment '**i**' value by one (**i++**). Repeat the same until '**i <= rear**' becomes **FALSE**.
- **Step 5:** If '**front <= rear**' is **FALSE**, then display '**queue[i]**' value and increment '**i**' value by one (**i++**). Repeat the same until '**i <= SIZE - 1**' becomes **FALSE**.
- **Step 6:** Set **i** to 0.
- **Step 7:** Again display '**cQueue[i]**' value and increment **i** value by one (**i++**). Repeat the same until '**i <= rear**' becomes **FALSE**.

Deque

➤ Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (**front** and **rear**). That means, we can insert at both front and rear positions and can delete from both front and rear positions.



- However, no element can be added and deleted from the middle. In the computer's memory, a deque is implemented using a circular doubly linked list.
- In a deque, two pointers are maintained, LEFT and RIGHT, which point to either end of the deque.
- The elements in a deque extend from the LEFT end to the RIGHT end and since it is circular, Dequeue[N-1] is followed by Dequeue[0].



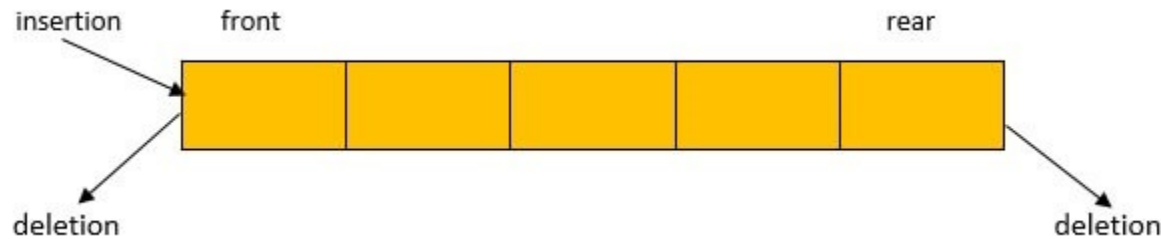


➤ Double Ended Queue can be represented in TWO ways, those are as follows...

- Input Restricted Double Ended Queue
- Output Restricted Double Ended Queue

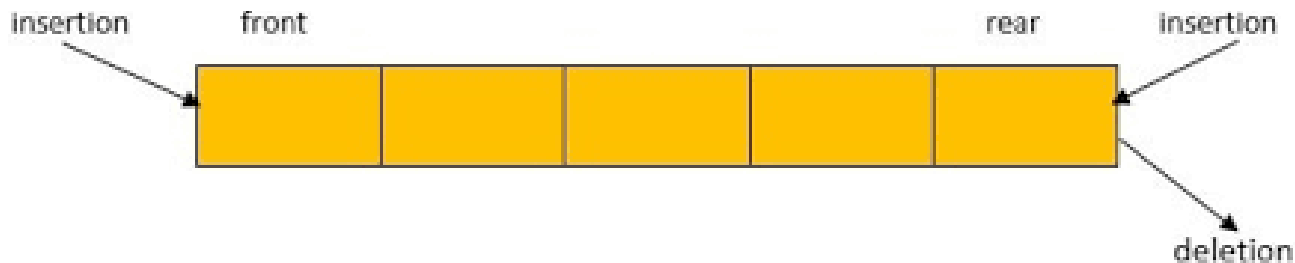
Input Restricted Double Ended Queue

- In this dequeue, insertions can be done only at one of the ends, while deletions can be done from both ends



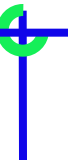
Output Restricted Double Ended Queue

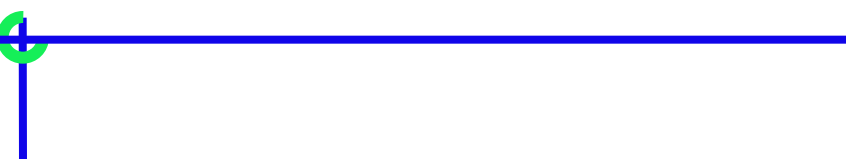
- In this dequeue, deletions can be done only at one of the ends, while insertions can be done on both ends

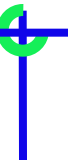


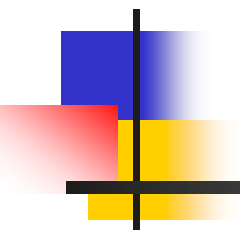
Priority Queue

- A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed.
- The general rules of processing the elements of a priority queue are -
 - An element with higher priority is processed before an element with a lower priority.
 - Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.

- 
-
- A priority queue can be thought of as a modified queue in which when an element has to be removed from the queue, the one with the highest-priority is retrieved first. The priority of the element can be set based on various factors.
 - Priority queues are widely used in operating systems to execute the highest priority process first. The priority of the process may be set based on the CPU time it requires to get executed completely.
 - For example, if there are three processes, where the first process needs 5 ns to complete, the second process needs 4 ns, and the third process needs 7 ns, then the second process will have the highest priority and will thus be the first to be executed.
 - However, CPU time is not the only factor that determines the priority, rather it is just one among several factors. Another factor is the importance of one process over another.

- 
- In case we have to run two processes at the same time, where one process is concerned with online order booking and the second with printing of stock details, then obviously the online booking is more important and must be executed first.

- 
-
- Priority Queue is an extension of queue with following properties.
 - 1) Every item has a priority associated with it.
 - 2) An element with high priority is dequeued before an element with low priority.
 - 3) If two elements have the same priority, they are served according to their order in the queue.
 - A typical priority queue supports following operations.
 - **insert(item, priority):** Inserts an item with given priority.
 - **getHighestPriority():** Returns the highest priority item.
 - **deleteHighestPriority():** Removes the highest priority item.

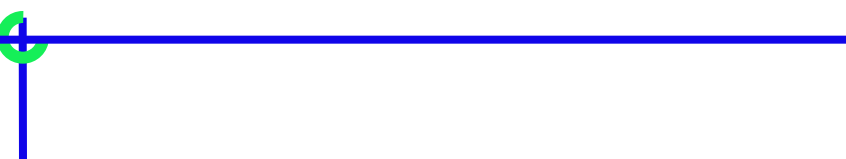


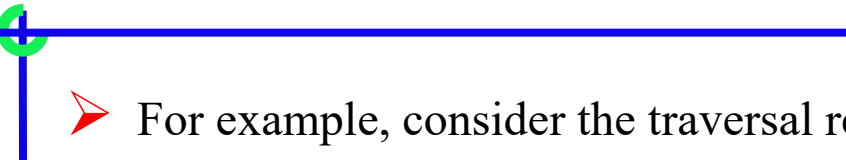
Tree





Constructing a Binary Tree from Traversal Results

- 
- We can construct a binary tree if we are given at least two traversal results. The first traversal must be the in-order traversal and the second can be either pre-order or post-order traversal.
 - The in-order traversal result will be used to determine the left and the right child nodes, and the pre-order/post-order can be used to determine the root node.

- 
- For example, consider the traversal results given below:

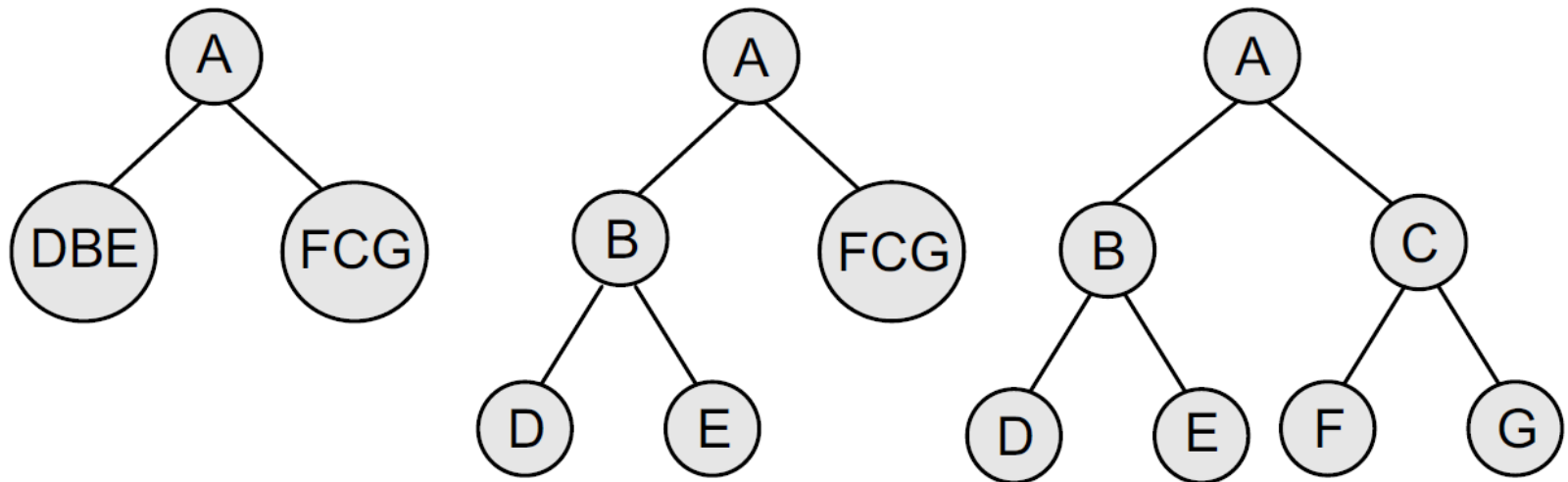
In-order Traversal: D B E A F C G

Pre-order Traversal: A B D E C F G

- Here, we have the in-order traversal sequence and pre-order traversal sequence. Follow the steps given below to construct the tree:
- **Step 1** Use the pre-order sequence to determine the root node of the tree. The first element would be the root node.
- **Step 2** Elements on the left side of the root node in the in-order traversal sequence form the left sub-tree of the root node. Similarly, elements on the right side of the root node in the in-order traversal sequence form the right sub-tree of the root node.
- **Step 3** Recursively select each element from pre-order traversal sequence and create its left and right sub-trees from the in-order traversal sequence.

In-order Traversal: D B E A F C G

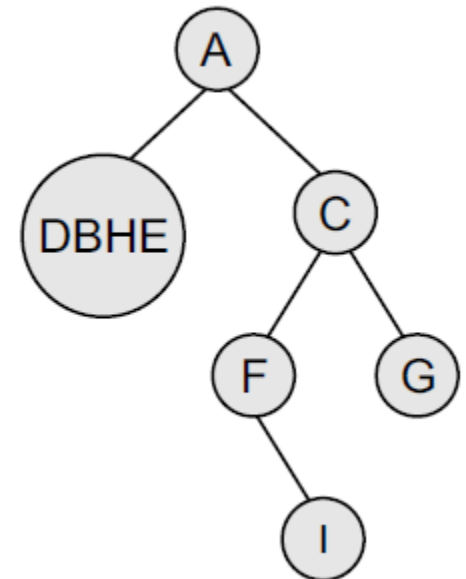
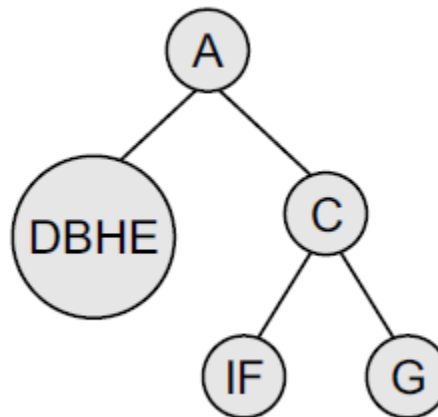
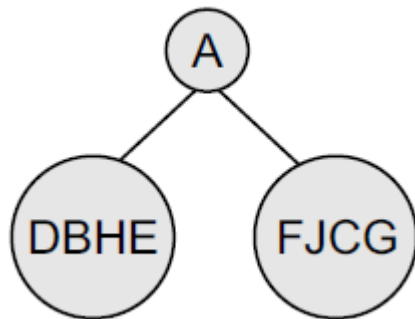
Pre-order Traversal: A B D E C F G

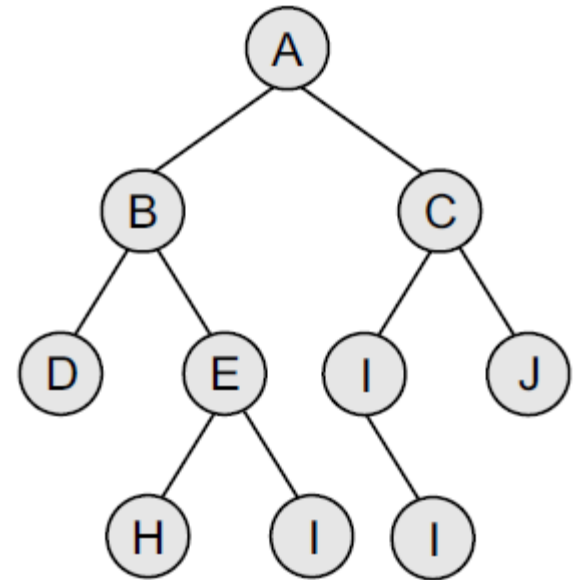
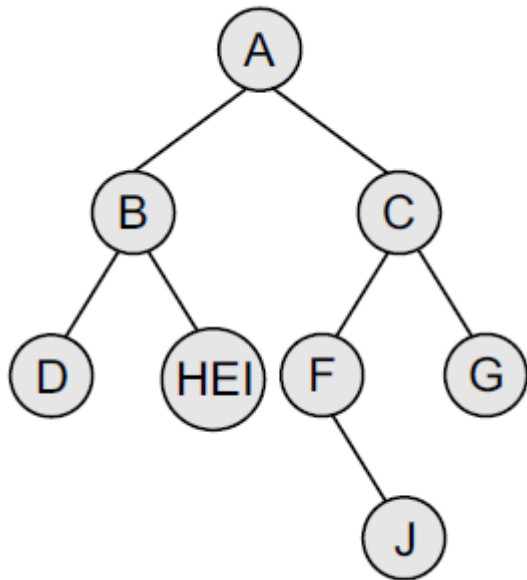


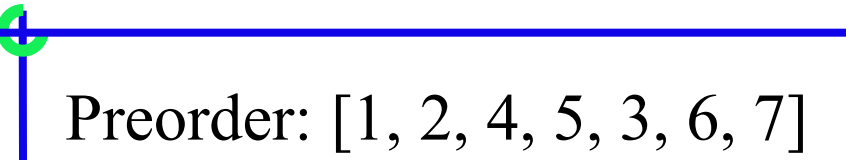
- Now consider the in-order traversal and post-order traversal sequences of a given binary tree. Before constructing the binary tree, remember that in post-order traversal the root node is the last node.

In-order Traversal: D B H E I A F J C G

Post order Traversal: D H I E B J F G C A





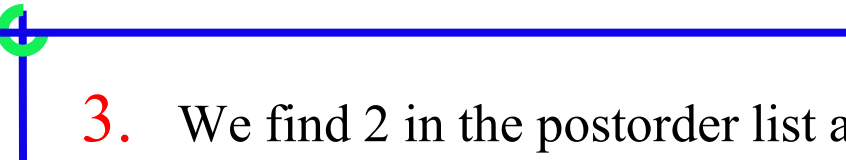


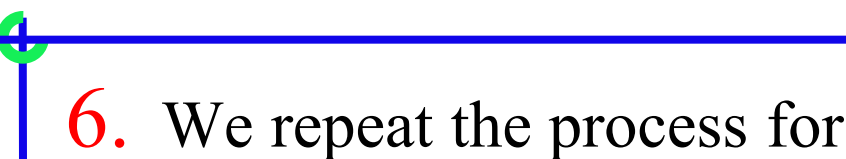
Preorder: [1, 2, 4, 5, 3, 6, 7]

Postorder: [4, 5, 2, 6, 7, 3, 1]

Using these two lists, we will walk through the construction of the binary tree step by step:

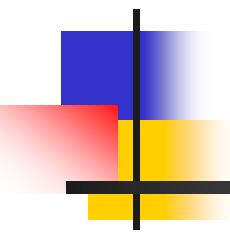
1. The first element from the preorder list is 1, which is the root node of the tree.
2. The second element in the preorder list is 2, which is the root of the left subtree.

- 
3. We find 2 in the postorder list at position 2. The elements before that (4, 5) are the left subtree's postorder.
 4. (a). For the left subtree: preorder sublist is [2, 4, 5] (skipping the first element 1 which is the root), and the postorder sublist is [4, 5, 2].
(b). For the right subtree: we take the remaining elements, preorder sublist [3, 6, 7] and postorder sublist [6, 7, 3].
 5. (a). Now, we will recursively construct the left subtree. The first element 2 becomes the root of the left subtree.
(b). Next, we look for 4 (second element of left preorder sublist) in the left postorder sublist. We find it at position 0. This means 4 is a leaf node, and similarly, 5 is a leaf node by the same process.

- 
6. We repeat the process for the right subtree. The first element 3 is the root. We look for 6 and 7 in the right postorder sublist and discover they are consecutive leaf nodes.
 7. We attach the constructed left subtree (2) with nodes 4 and 5 as leaf children, and right subtree (3) with nodes 6 and 7 as leaf children, to the root node 1.
 8. The recursion continues until all nodes are placed correctly with their respective children, and we obtain the final binary tree.

- 
- By following the steps of the recursion according to the preorder and postorder lists, we've successfully rebuilt the binary tree:

```
    1
   /\
  2 3
 /\ /\
4 5 6 7
```



Binary Search Tree



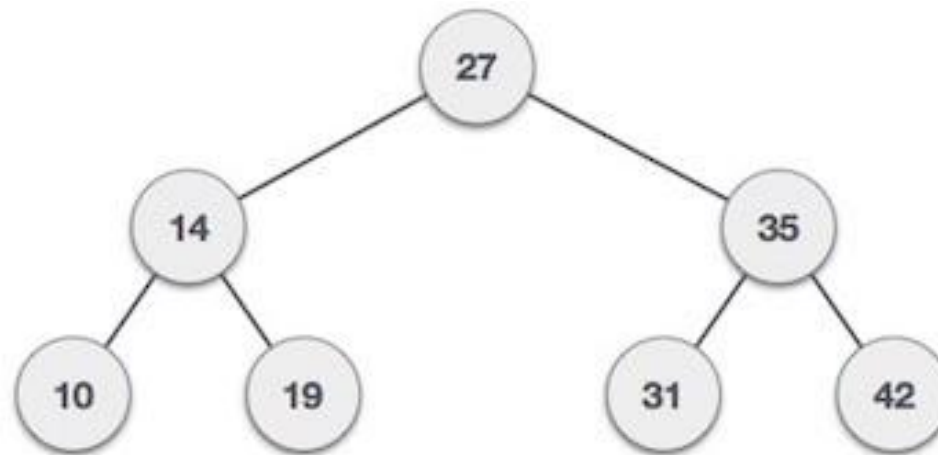
Binary Search tree

- A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties –
 - The left sub-tree of a node has a key less than or equal to its parent node's key.
 - The right sub-tree of a node has a key greater than to its parent node's key.
- Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right sub-tree and can be defined as –

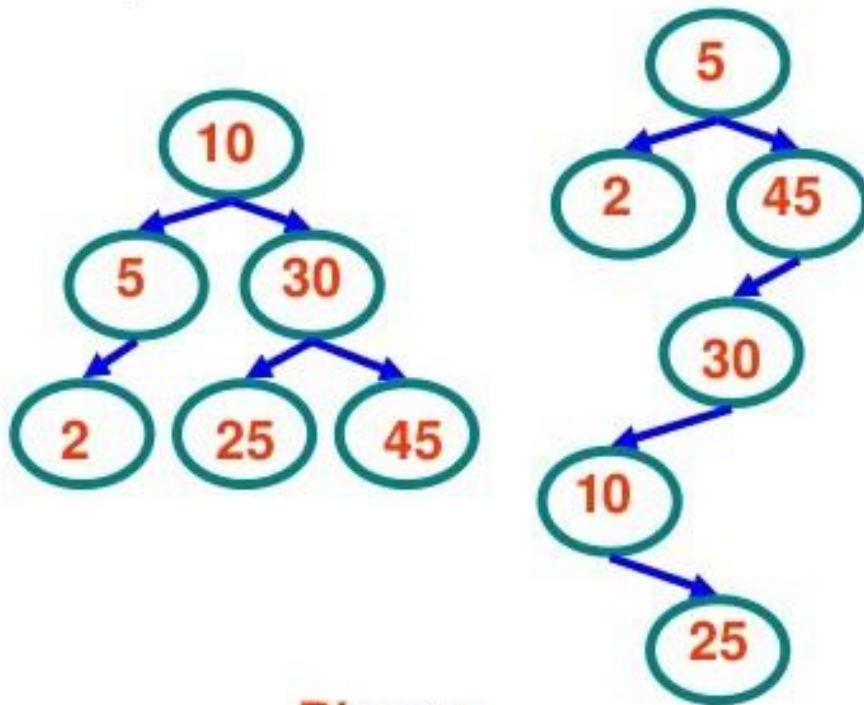
$$\text{left_subtree (keys)} \leq \text{node (key)} \leq \text{right_subtree (keys)}$$

BST Representation

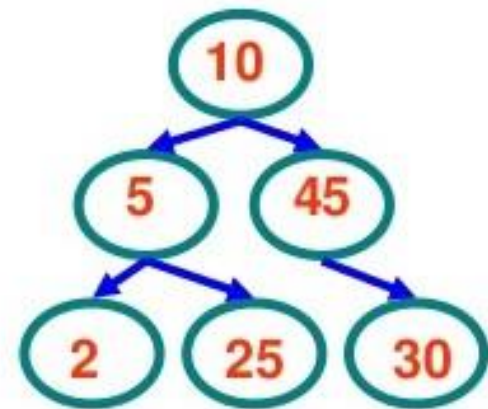
- BST is a collection of nodes arranged in a way where they maintain BST properties. Each node has a key and an associated value. While searching, the desired key is compared to the keys in BST and if found, the associated value is retrieved.



Examples



**Binary
search trees**

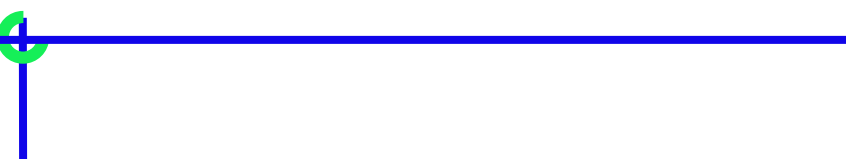


**Not a binary
search tree**

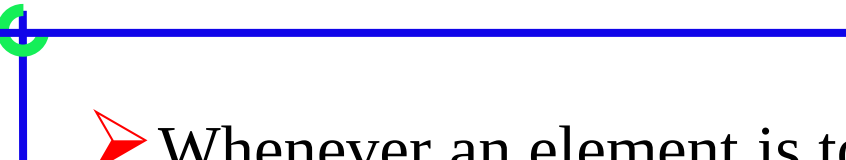
Difference between BT and BST

- A binary tree is simply a tree in which each node can have at most two children.
- A binary search tree is a binary tree in which the nodes are assigned values, with the following restrictions.
 - No duplicate values.
 - Left subtree of a node can only have values less than the node.
 - Right subtree of a node can only have values greater than node.
 - The left and right subtree of a node is a binary search tree.

Operations in BST

- 
- Following are the basic operations of a tree –
 - **Search** – Searches an element in a tree.
 - **Insert** – Inserts an element in a tree.
 - **Pre-order Traversal** – Traverses a tree in a pre-order manner.
 - **In-order Traversal** – Traverses a tree in an in-order manner.
 - **Post-order Traversal** – Traverses a tree in a post-order manner.

Search Operation

- 
- Whenever an element is to be searched, start searching from the root node. Then if the data is less than the key value, search for the element in the left subtree. Otherwise, search for the element in the right subtree. Follow the same algorithm for each node.

➤ Search for a matching node

- 1. Start at the root node as current node
- 2. If the search key's value matches the current node's key then found a match
- 3. If search key's value is greater than current node's
 - 1. If the current node has a right child, search right
 - 2. Else, no matching node in the tree
- 4. If search key is less than the current node's
 - 1. If the current node has a left child, search left
 - 2. Else, no matching node in the tree



```
struct node* search(int data){
    struct node *current = root;
    printf("Visiting elements: ");
    while(current->data != data){

        if(current != NULL) {
            printf("%d ",current->data);

            //go to left tree
            if(current->data > data){
                current = current->leftChild;
            }//else go to right tree
```

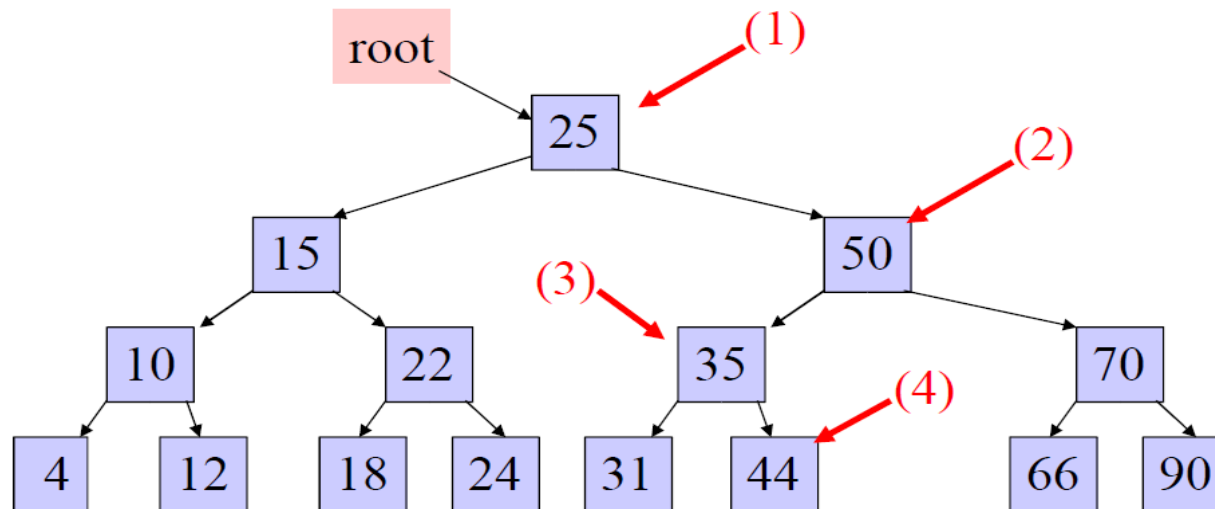
```
        else {
            current = current->rightChild;
        }

        //not found
        if(current == NULL){
            return NULL;
        }
    }
    return current;
}
```

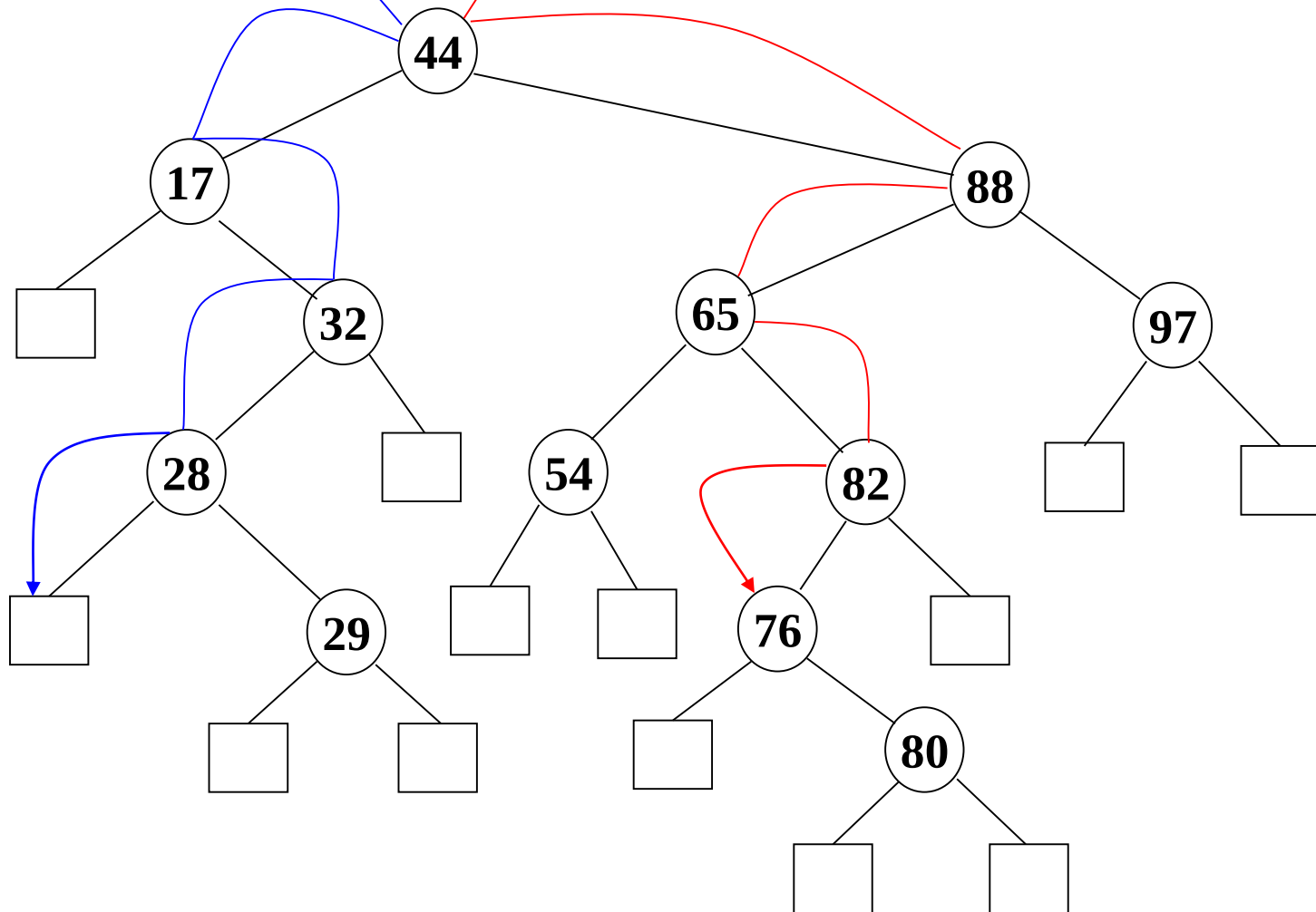
Example: search for 45 in the tree

(key fields are show in node rather than in separate obj ref to by data field):

1. start at the root, 45 is greater than 25, search in right subtree
2. 45 is less than 50, search in 50's left subtree
3. 45 is greater than 35, search in 35's right subtree
4. 45 is greater than 44, but 44 has no right subtree so 45 is not in the BST



➤ Search: Search(25) Search(76)



Insert Operation

- 
- Whenever an element is to be inserted, first locate its proper location. Start searching from the root node, then if the data is less than the key value, search for the empty location in the left subtree and insert the data. Otherwise, search for the empty location in the right subtree and insert the data.

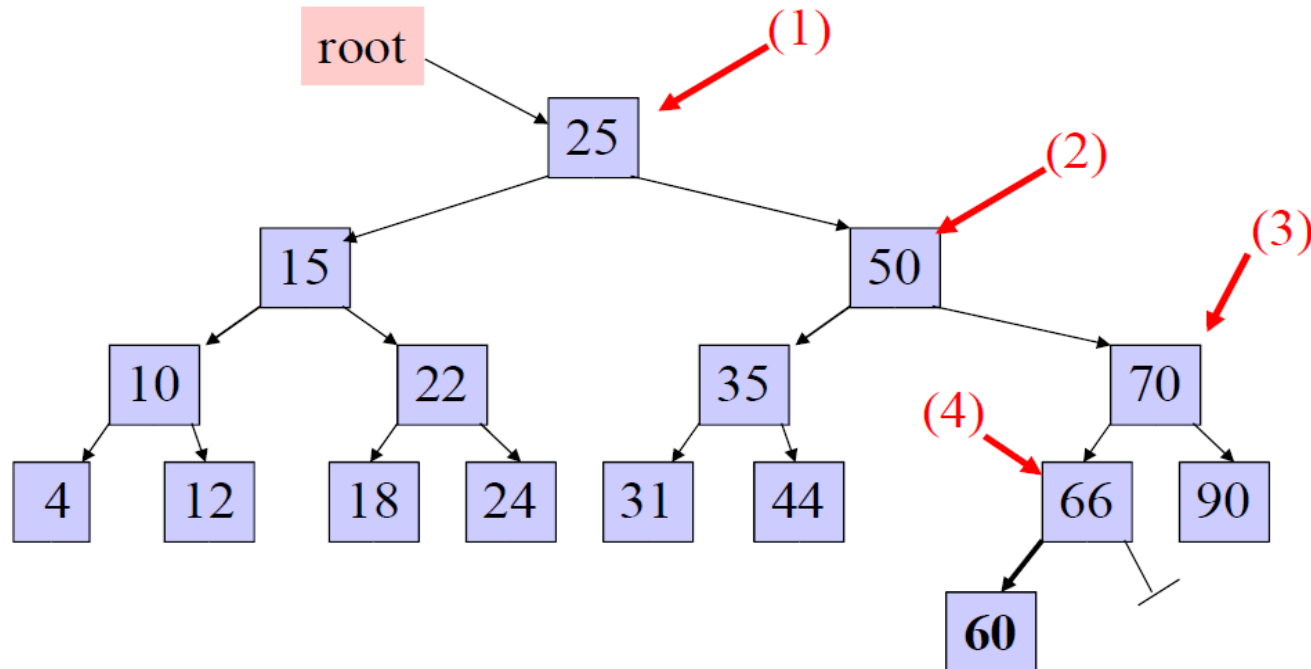


➤ Always insert new node as leaf node

- 1. Start at root node as current node
- 2. If new node's key $<$ current's key
 - 1. If current node has a left child, search left
 - 2. Else add new node as current's left child
- 3. If new node's key $>$ current's key
 - 1. If current node has a right child, search right
 - 2. Else add new node as current's right child

Example: insert 60 in the tree:

1. start at the root, 60 is greater than 25, search in right subtree
2. 60 is greater than 50, search in 50's right subtree
3. 60 is less than 70, search in 70's left subtree
4. 60 is less than 66, add 60 as 66's left child



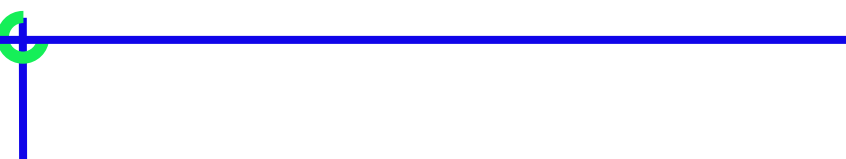
Traversal Operation

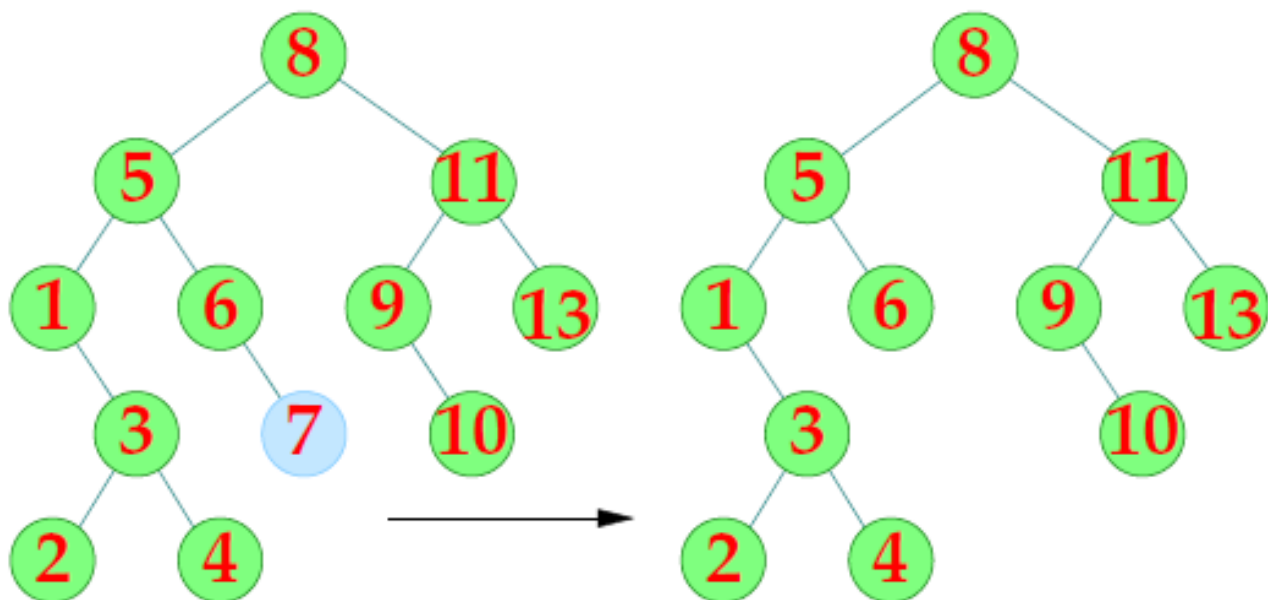


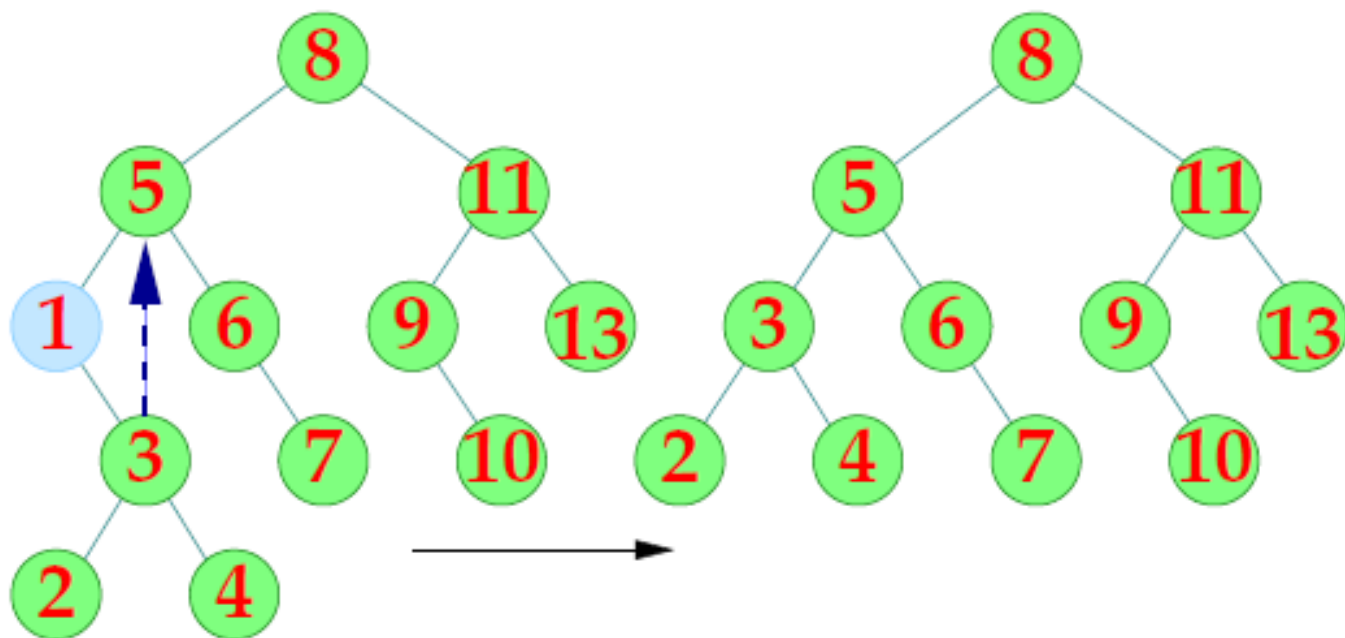
➤ Same as normal Tree Traversal

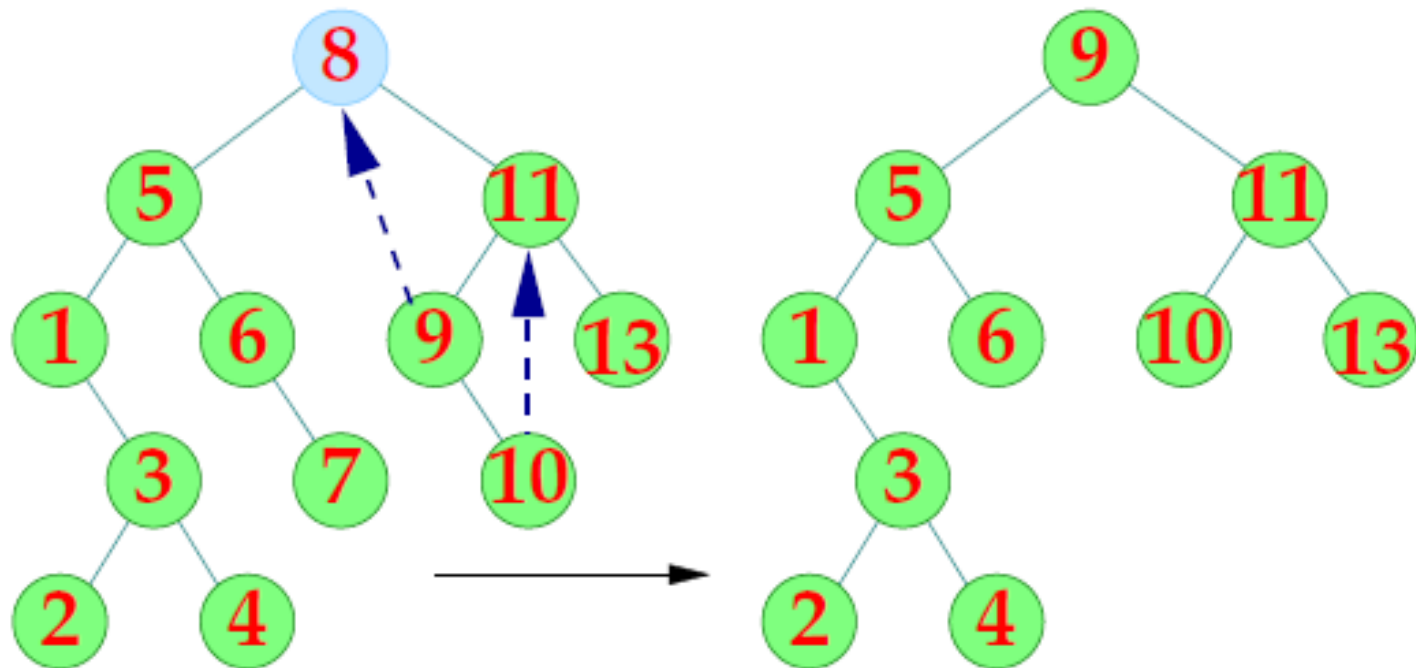
- Pre order
- In-order
- Post order

Deletion Operation

- 
- Suppose we want to delete a node z .
- 1. If z has no children, then we will just replace z by nil .
 - 2. If z has only one child, then we will promote the unique child to z 's place.
 - 3. If z has two children, then we will identify z 's successor. Call it y . The successor y either is a leaf or has only the right child. Promote y to z 's place. Treat the loss of y using one of the above two solutions.







➤ BST-Delete(T, z)

- 1: if $\text{left}[z] = \text{nil}$ or $\text{right}[z] = \text{nil}$
- 2: then $y \leftarrow z$
- 3: else $y \leftarrow \text{BST-Successor}(z)$
- 4: ✂ y is the node that's actually removed.
- 5: ✂ Here y does not have two children.
- 6: if $\text{left}[y] \neq \text{nil}$
- 7: then $x \leftarrow \text{left}[y]$
- 8: else $x \leftarrow \text{right}[y]$
- 9: ✂ x is the node that's moving to y 's position.
- 10: if $x \neq \text{nil}$ then $p[x] \leftarrow p[y]$
- 11: ✂ $p[x]$ is reset If x isn't NIL.
- 12: ✂ Resetting is unnecessary if x is NIL.

- 
- 13: if $p[y] = \text{nil}$ then $\text{root}[T] \leftarrow x$
 - 14: ✂ If y is the root, then x becomes the root.
 - 15: ✂ Otherwise, do the following.
 - 16: else if $y = \text{left}[p[y]]$
 - 17: then $\text{left}[p[y]] \leftarrow x$
 - 18: ✂ If y is the left child of its parent, then
 - 19: ✂ Set the parent's left child to x .
 - 20: else $\text{right}[p[y]] \leftarrow x$
 - 21: ✂ If y is the right child of its parent, then
 - 22: ✂ Set the parent's right child to x .
 - 23: if $y \neq z$ then
 - 24: { $\text{key}[z] \leftarrow \text{key}[y]$
 - 25: Move other data from y to z }
 - 26: return (y)



➤ Height of a binary search tree

- The height of a binary search tree with n elements can become as large as n .
- It can be shown that when insertions and deletions are made at random, the height of the binary search tree is $O(\log_2 n)$ on the average.
- Search trees with a worst-case height of $O(\log_2 n)$ are called *balance search trees*

Ranked Binary Search Tree

- In ranked binary search tree, each node of the tree has an additional field name “leftsize”, which is one plus number of elements in the left subtree of the node.
- The algorithm searches for the K-th smallest element in ranked tree in $O(h)$ time where h means height of the tree.

Algorithm Search K(k)

```
{  
  Found=false  
  t=tree  
  while((t ≠ 0) and not found) do  
  {  
    if (k=(t → leftsize)) then  
      found=true  
    else if (k<(t → leftsize)) then  
      t = t → l_child  
    else
```

```
{  
  k=k - (t → leftsize)  
  t = t → r_child  
}  
}  
if (not found) then  
  return 0  
else  
  return 1  
}
```



Threaded Binary Tree

Binary Tree

Do you find any drawback of the previous tree?

Too many null pointers in current representation of binary trees

n : number of nodes

number of non-null links: $n-1$

total links: $2n$

null links: $2n - (n-1) = n+1$